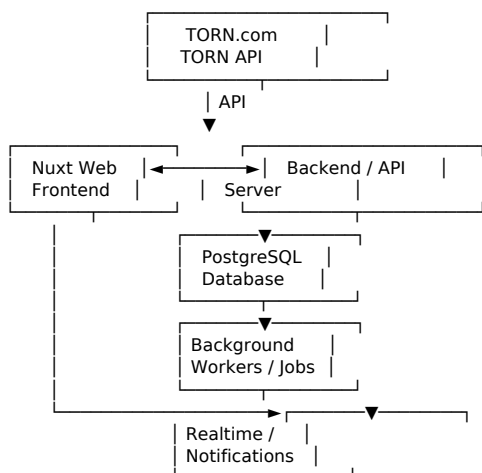


TORN: The Contract — Backend Architecture

1. High-Level Architecture

A good production architecture would look like this:



Frontend: Nuxt
Backend API: Nuxt server routes / separate API service
Database: PostgreSQL via Supabase
Authentication: Supabase Auth
Realtime: Supabase Realtime / WebSockets
Scheduled jobs: cron/worker service
Hosting: Vercel
Background processing: Railway/worker service if needed
External data: TORN API

2. The Database Is the Source of Truth

Don't store hardcoded values such as Total Users = 12,482, Active Contractors = 482, Contract #1847, or PlayerA = 9/10 in the frontend. Those are calculated values.

Users

users

id
torn_user_id
torn_username
torn_level
created_at
last_seen_at
status

Total Registered Users is then simply the count of users.

3. Contracts Table

The contract itself should be a database record.

contracts
id
status
type
starts_at
ends_at
max_objectives
winner_count
created_by
created_at
reward_configuration
settings
generation_method
published_at
completed_at

4. Contract Objectives

Don't store objectives directly inside the contract as static text. Create an objective library.

```

objective_definitions
id | name | category | difficulty

contract_objectives
id
contract_id
objective_definition_id
objective_order
target_value
verification_status
created_at
Contract #1847
├── Objective #1 → Use 1,500 Energy
├── Objective #2 → Fly to Switzerland twice
├── Objective #3 → Lose 5 attacks
├── Objective #4 → Defeat player 5+ levels above
└── ...

```

5. Player Contract Participation

Create a table representing which user participated in which contract.

```

contract_participants
id
contract_id
user_id
started_at
completed_at
status
current_rank
progress
last_activity_at

```

The dashboard then calculates the user's position from actual participants.

6. Individual Objective Progress

```

participant_objectives
id
participant_id
contract_objective_id
status
current_value
target_value
started_at
completed_at
verified_at
verification_attempts
PlayerA / Contract #1847

```

```

Objective 1: 1500 / 1500 → VERIFIED
Objective 2: 1 / 2 → IN_PROGRESS
Objective 3: 5 / 5 → VERIFIED

```

7. Don't Poll TORN for Everything

You don't want every user hitting the TORN API every second. Use a verification/event pipeline.

```

TORN
↓
API / available data
↓
Background Worker
↓
Detect relevant activity
↓
Store event
↓
Verify objective
↓
Update participant objective
↓
Update contract progress
↓
Realtime update
↓
Frontend

```

8. Activity/Event Table

```

player_events
id

```

user_id
event_type
event_time
source
payload
created_at

Examples: ENERGY_USED, FLIGHT_COMPLETED, ATTACK_WON, ATTACK_LOST, CRIME_COMPLETED, ITEM_SOLD, ITEM_PURCHASED, MEDICAL_ITEM_USED.

9. Objective Verification Engine

This should be its own backend service/module, the Contract Engine. Its job is to determine whether a player has completed an objective.

Player level = 50
Opponent level = 56
Attack result = WIN
→ Objective VERIFIED
Use 1,500 Energy
Current verified energy used = 1,250
→ IN PROGRESS

Current = 1,500
→ VERIFIED

10. Verification Should Be Idempotent

If the same TORN event is processed twice, duplicate progress must not occur. Give every event a unique identifier and prevent duplicate processing at the database level.

11. Contract Completion

Objective 10
↓
VERIFIED
↓
Are all 10 objectives verified?
↓
YES
↓
Participant = COMPLETED
↓
Record completed_at
↓
Calculate ranking
↓
Determine winner

The database transaction should handle this atomically.

12. Handling Simultaneous Winners

For Standard = 1 winner, Top 3 = 3 winners, Top 10 = 10 winners, use server-side transactional ordering. Do not trust browser timestamps.

PlayerA completes at 14:32:10.241
PlayerB completes at 14:32:10.243
↓
transactional completion sequence
↓
PlayerA = #1
PlayerB = #2

13. Leaderboard Should Be Calculated

The frontend shouldn't store PlayerA = #1, PlayerD = #2, PlayerB = #3. Calculate rankings from contract participants using progress, completion status, and completion time. Caching is fine, but the database remains the source of truth.

14. Admin Dashboard

Total Registered Users = COUNT(users)

Active Contractors =
COUNT(contract_participants for active contract)

Contract Progress =

COUNT(participant_objectives where status = verified)

Total Rewards =
SUM(reward_transactions.amount)

15. Reward System

reward_definitions
id
name
type
value
metadata

reward_transactions
id
user_id
contract_id
reward_definition_id
status
issued_at
delivered_at
error

Possible states: PENDING, PROCESSING, DELIVERED, FAILED, CANCELLED.

16. Archive Becomes Automatic

Contract
↓
COMPLETED
↓
Winner determined
↓
Rewards processed
↓
Contract becomes immutable
↓
Archive automatically displays it

17. Realtime Updates

When PlayerA completes objective #10, the backend verifies it, determines the winner, updates the database, and emits a realtime event. Everyone viewing Contract #1847 can see the completion and leaderboard update without refreshing.

18. Background Jobs

Contract generation: Generate the daily contract, select objectives, calculate rewards, and publish it.

TORN synchronization: Fetch relevant TORN data, process events, and verify objectives.

Contract cleanup: Finalize expired contracts, determine winners, and process rewards.

Health checks: Monitor TORN API, database, workers, and reward system.

19. Job Queue

Database
↓
Job Queue
↓
Workers
├── TORN Sync Worker
├── Objective Verification Worker
├── Reward Worker
├── Contract Worker
└── Notification Worker

This is more reliable than trying to make everything happen inside Nuxt API requests.

20. Admin Actions Need an Audit Log

admin_audit_logs
id
admin_id
action
entity_type
entity_id
old_value
new_value

ip
created_at

Record contract changes, objective changes, manual verification, reward actions, user suspension, bans, and system configuration changes.

21. Suspicious Activity

security_events
id
user_id
event_type
severity
description
metadata
created_at
resolved_at

Examples: DUPLICATE_EVENT, RAPID_OBJECTIVE_COMPLETION, INVALID_API_RESPONSE, MULTIPLE_ACCOUNT_PATTERN, MANUAL_VERIFICATION.

22. Authentication

User
↓
Authenticate
↓
Verify TORN identity
↓
Create/link local account
↓
Store torn_user_id

Use the stable TORN user ID rather than usernames, since usernames can change.

23. TORN API Keys

If users provide TORN API keys, don't expose them to the frontend unnecessarily. Use the backend as the secure intermediary and store sensitive credentials encrypted or in an appropriate secrets system.

24. Suggested Database Structure

```
users
├── user_sessions
├── player_events
├── contract_participants
│   └── participant_objectives
├── security_events
└──

contracts
├── contract_objectives
│   └── objective_definitions
├── contract_participants
├── reward_transactions
├── contract_events
└──

objective_definitions
├── objective_verification_rules
└──

reward_definitions
├── reward_transactions
└──

admin_users
├── admin_audit_logs
└──

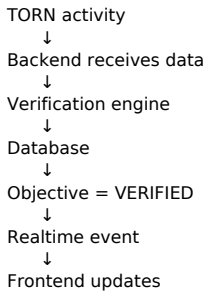
system
├── announcements
├── jobs
├── api_logs
├── system_alerts
└──
```

25. What the Frontend Actually Does

The Nuxt frontend asks the backend for dashboard data. The backend returns platform statistics, contract information, user progress, leaderboard, alerts, announcements, and activity. Nuxt simply renders it.

26. The Most Important Principle

The frontend never decides whether something happened. The backend verifies it.



That makes the system much harder to manipulate.

27. Recommended Stack For Your Project

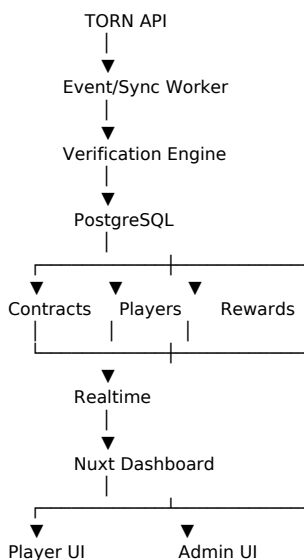
Frontend: Nuxt
UI: Nuxt UI
Backend API: Nuxt Server API
Database: PostgreSQL
DB platform: Supabase
Authentication: Supabase Auth
Realtime: Supabase Realtime
Scheduled jobs: Cron
Heavy workers: Railway
Cache: Redis, when needed
External API: TORN API
Hosting: Vercel
Monitoring: Sentry + server logs

Initially you don't need Redis, a complicated microservice architecture, or Kubernetes. Start with Nuxt + Supabase/Postgres + Nuxt Server API + Supabase Realtime + Cron/Worker + TORN API. Introduce Redis/queues/workers when actual traffic requires them.

28. Recommended Development Order

Phase 1 — Foundation: Authentication, Users, PostgreSQL schema, Admin roles, Basic API.
Phase 2 — Contract Engine: Objective library, Contract creation, Random objective generation, Contract lifecycle, Contract participation.
Phase 3 — Verification: TORN API integration, Event ingestion, Objective verification, Progress tracking, Completion detection.
Phase 4 — Competition: Ranking, Winner determination, Multiple winners, Tie handling, Contract archive.
Phase 5 — Rewards: Reward definitions, Reward transactions, Delivery system, Failed reward handling.
Phase 6 — Realtime: Live progress, Live leaderboard, Activity feed, Notifications.
Phase 7 — Admin: Admin dashboard, Objective management, Contract management, Participant inspection, Verification controls, Reward management, Audit logs.
Phase 8 — Reliability: Background workers, Job queues, Retry mechanisms, API rate-limit handling, Monitoring, Error tracking, Suspicious activity detection.

The End Result



The database owns the state, the backend owns the rules, the verification engine owns objective completion, and Nuxt only presents the resulting state.